MatchPulse — Methodology Report

Date	2026-11-08
Status	Current project report; the semester is still running.
Scope	Development Methodologies (CM0639, module 2), shared with Software Architectures.
Repository	https://github.com/match-pulse/match-pulse

This is a dated report, not a final submission. It curates the project's methodology records into one narrative: the claim, the operating model, the quality evidence, the architecture change, and the delivery/recovery story. The detailed records remain authoritative and are linked throughout.

1. Project and claim

MatchPulse is a football tournament platform for local competitions. Spectators buy tickets, register for a place, follow live match data and trade prediction points on match outcomes. Points are redeemable for ticket discounts, and aggregated prediction data is sold to clubs.

The bounded first increment is **capacity-safe tournament registration plus the live prediction market**. The claim is:

> For a tournament organizer running a local competition, the platform will keep participation at or below the configured capacity and keep point balances, positions and trades consistent when many spectators act at the same moment, measured by a concurrent burst and committed-state assertions against a real PostgreSQL database, with the limitation that the evidence is local/CI, uses a single database instance, and does not yet include real-user observation.

The claim is deliberately narrow. It does not claim that the whole product vision is implemented, that the system is production-ready, or that the crowd data has been validated with clubs.

2. Context and stakeholders

Stakeholder	Interest	Influence	Decision rights	Exposure
Spectators	Buy tickets, register, trade, redeem rewards	Low individually, high collectively	Their own actions only	Lost points, refused place, confusing market
Tournament staff	Run the tournament, set capacity, finalize results	High for the event	Match setup, capacity, result finalization	Overbooking, disputed result, manual work

Stakeholder	Interest	Influence	Decision rights	Exposure
Club representatives	Aggregated crowd data	Medium	Read/export only	Privacy breach, misleading aggregate
Project owner	Deliver a coherent project for both courses	High	Product, architecture, release, risk acceptance	Scope, time, unverified claims
Course reviewers	Assess reasoning, evidence and limits	High for the grade	Accept/reject the project claims	Being misled by an overclaim

The organizer is the primary user for the first increment. The problem claim is still a hypothesis: no organizer has been interviewed or observed yet (FND-007). The project states that limitation rather than presenting the hypothesis as validated.

3. Operating model and lifecycle

The project uses a **hybrid lifecycle**:

- **Plan-driven domain contract:** the invariants, authorization rules and release policy are fixed constraints. They are not renegotiated per sprint.
- **Iterative and incremental delivery:** small end-to-end increments (capacity, market, authorization, idempotency, delivery) are built, tested and released.
- **Adaptive scope:** evidence can change the next increment; the checkpoint records and findings register feed the backlog.

This fits the dominant uncertainty: the product direction is bounded, but the implementation and evidence are discovered incrementally. The lifecycle decision is recorded in `checkpoint-B/04-sdlc-decision.md`.

Work flows through GitHub issues, a short-lived branch, a pull request, CI and a review decision. The project board uses four stages: Enter, Build, Review Queue, Release & Learn. The flow evidence is derived from the repository history in `checkpoint-B/03-flow-evidence.md`; it is one developer's history, not a team benchmark.

4. Checkpoint A — context, claims and evidence

Checkpoint A established the bounded proposal, the system invariants, the product claims and the governance model. The first version of the capacity claim was reopened because the evidence could not fail. The corrected evidence uses committed-state assertions and fault injection.

Key records:

- `checkpoint-A/01-bounded-proposal.md`
- `checkpoint-A/02-invariants.md`
- `checkpoint-A/03-product-claims.md`
- `checkpoint-A/04-governance.md`

The invariant registry is the authoritative status of every rule. It distinguishes **enforced**, **verified** and **enforced, unverified**. The distinction is the core of the project's evidence discipline.

5. Checkpoint B — operating model and feedback

Checkpoint B records the system dynamics, the lifecycle choice, the flow evidence and the decision rights. It also records a correction: the operating model claimed continuous integration protected domain correctness, but CI originally ran only lint and build. The invariant job was added in #60. The documentation asserted a control that did not exist; the correction is part of the record.

Key records:

- `checkpoint-B/01-system-dynamics.md`
- `checkpoint-B/02-operating-model.md`
- `checkpoint-B/03-flow-evidence.md`
- `checkpoint-B/04-sdlc-decision.md`

6. Checkpoint C — quality and residual risk

Checkpoint C answers the four questions:

1. **Which risk matters most, and why?** The ranking changed over time. Initially it was unauthorized access/resolution (R-01/FND-001); after authentication and RLS, the top verification risk became the un-injected market invariants (FND-002), then delivery/operations (FND-006, now partly fixed). The current ranking and evidence are in `checkpoint-C/07-current-state.md`.
2. **What behaviour was tested, and what happened?** The project tested the double-spend path, terminal resolution, the cooldown gate and the capacity burst, and observed them failing under deliberate weakening. The quality story is in `checkpoint-C/02-quality-story.md`.
3. **What did review change, or deliberately keep?** Review changed the lock order in `close_trade` and added the cooldown-exit evidence while keeping the code. The review decisions are in `checkpoint-C/03-review-decision.md`.
4. **What remains uncertain, and what happens next?** The residual risk register is `checkpoint-C/05-residual-risk.md`. FND-003 is fixed; FND-004, FND-006 and FND-002 are partly fixed; FND-007 and FND-008 remain open. FND-002 is partly fixed because the matrix covers 10 observed-red mutations and the M-INV2 forced-failure rollback, but M-INV2 cannot be observed-red without a second connection.

The strongest quality evidence is not a coverage number. It is the combination of a business-rule oracle, a real database, committed-state assertions and observed failure under fault injection.

7. Checkpoint D — architecture and changeability

Checkpoint D is prepared as a pre-read because the L13–L16 materials are not yet in the course folder. The preparation includes:

- ownership and change rights;
- dependency rules and executable fitness functions;
- a realistic compatible change: request-key idempotency;
- compatibility and evolution policy;
- technical debt and evolution.

The idempotency change is implemented and verified. It added a `trade_requests` table, claim-before-work in `open_trade` / `close_trade`, client request-key retention, replay/refusal tests, and an architecture-guard update. The change is recorded in `checkpoint-D/03-change-scenario-idempotency.md` and delivered in #107 / PR #108 / release `v1.2.0`.

The pre-read is `checkpoint-D/README.md`. It is not the final Checkpoint D deliverable; the actual L13–L16 questions will determine the final shape.

8. Checkpoint E — delivery, observation and recovery

Checkpoint E is also a pre-read. The project now has a real local/CI delivery baseline:

- `GET /api/health` returns app/database status without sensitive data.
- JSON logs record health failures and key action failures.
- `bun run smoke` checks health, the login UI and the home shell.
- the CI `Delivery and recovery` job starts Supabase, applies migrations, builds, starts the app, checks health, runs smoke, and exercises a backup/restore drill.
- the operations runbook covers release, rollback, backup/restore and incident steps.
- proposed SLOs and a p95 measurement method are recorded.
- the capacity burst records p50/p95/max; observed p95 is 463.11 ms and 571.68 ms.
- the recovery drill restored a 67,009-byte public-schema backup into a scratch database: tables 10→10, RLS policies 10→10, tournaments 2→2, marker present.

The evidence is in `checkpoint-E/README.md` and the delivery verification record and recovery verification record.

The limits are explicit: no hosted deployment, external monitoring/alerting or production recovery; the backup covers the application-owned `public` schema, not Supabase-managed auth/storage data; p95 is an observation, not a hard gate.

9. Architecture and changeability in practice

The architecture is a modular monolith: one Next.js application, one PostgreSQL database, domain modules in the schema/RPC surface, and a layered delivery shell. The critical rules are enforced in the database because the domain requires immediate, unambiguous answers under concurrent requests.

The idempotency change is the best current example of changeability:

- **Compatibility:** the request ID is optional in the RPC signature, so old callers can still call the function; the application always sends a key.
- **Data:** the new table is additive; no destructive migration.
- **Tests:** six new M-INV9 tests plus authorization coverage for `trade_requests`.
- **Guard:** the architecture guard scans the latest migration definitions and exact RPC signatures.
- **Release:** `v1.2.0`, with migration notes and known gaps.

The change did not alter pricing, settlement or authorization rules. It closed FND-003 and left the legacy null-key path documented as non-idempotent.

10. Evidence index

Claim	Evidence	Limit
Capacity and uniqueness hold under a burst	`tests/invariants/tournament-capacity.test.ts`, `registration-uniqueness.test.ts`; fault injection	Local/CI, one database
Double-spend is prevented	`market-concurrency.test.ts`; observed-red injection	Single instance
Resolution is one-shot	`market-lifecycle.test.ts`; observed-red injection	Local/CI
Retries do not duplicate trades	`idempotency.test.ts`; six M-INV9 tests	Legacy null-key path
Authorization boundary	`authorization.test.ts`; RLS/grants	No production hardening; club privacy rule not implemented
Burst latency	capacity burst p50/p95/max	Environment-specific; post-peak resource release unverified
Delivery	`Delivery and recovery` CI job; smoke; runbook	Local/CI only
Recovery	public-schema backup/restore drill	Application schema only; no production RTO/RPO
Tests can fail	Fault-injection matrix: 10 observed-red mutations plus the M-INV2 forced-failure rollback	M-INV2 is not observed-red; local/CI only

11. Review, AI use and accountability

The project is one developer plus agent role passes. AI contributed implementation, tests, documentation, review passes and release preparation. The human project owner approved the plans, domain rules, architecture decisions, releases and residual risk. The AI transparency log records each contribution, verification and limitation.

AI-generated code and tests can share assumptions, so the project does not treat a green AI-generated suite as proof. The business-rule oracle, the observed-red fault injections and the

human decisions are the checks. The project also records what AI got wrong and what was corrected.

The remaining review gap is FND-008: there is no independent human review. The review evidence is structured self-review plus automated checks and agent role passes. A peer review of one PR would close that gap.

11.1 OpenSpec adoption (2026-11-08)

OpenSpec was installed as a standalone CLI (`openspec 1.16.1`) and initialized in the repository. The first bounded trial is the change `add-market-fault-injections`, which targets the remaining fault-injection gap (FND-002).

- The change has a proposal, a design with a mutation catalog, and a tracked task list. It sets `skip_specs: true` because it adds verification evidence, not system behavior.
- `openspec validate add-market-fault-injections --type change --strict` passes.
- The change was implemented and verified in CI run 42474762811: 10 observed-red mutations pass, the M-INV2 forced-failure rollback test passes, and the full invariant suite returns to 28 pass / 0 fail.
- M-INV2 is not observed-red; PostgreSQL function atomicity cannot be broken without a second connection. FND-002 remains partly fixed for that reason.
- OpenSpec is a change-planning layer, not a second source of truth: the kata, domain model, invariant registry, ADRs, tests and verification records remain authoritative.
- The change is ready to archive after the documentation and trial review.

The adoption decision and its limits are recorded in `../development-practices.md` §4.6 and `../../openspec/README.md`. The evidence is in the fault-injection verification record.

12. Residual risk and next actions

Finding	Status	Next action
FND-002 — remaining market fault injections	Partly fixed	10 observed-red mutations are recorded; M-INV2 is verified by forced-failure rollback and cannot be observed-red without a second connection. Keep the limit explicit.
FND-004 — load/p95	Partly fixed	Keep the p95 observation current; add sustained load and a hard gate only when the environment supports it
FND-006 — delivery/operations	Partly fixed	Keep local/CI evidence current; decide whether a hosted deployment is required
FND-007 — real organizer evidence	Open	Interview or observe one organizer/fan; add a dated addendum to the claim
FND-008 — independent review	Open	Ask a peer to review one PR with a checklist; record the decision

Checkpoints D and E remain pre-reads until L13–L16 and L17–L20 are available. The project-side evidence is ready to be mapped to the actual questions when the materials arrive.

13. Lessons

- A green test is not evidence unless it can fail. Fault injection changed the project's claims more than any passing run.
- The documentation can assert a control that does not exist. CI originally ran lint and build but no domain test; the record was corrected.
- A retry is not a new business action. Request-key idempotency closed a gap the cooldown could not cover.
- A health endpoint must not depend on a user session; the first implementation logged a false database failure during prerender and was corrected.
- A backup is not a recovery until it has been restored and verified.
- The course vocabulary matters: calling a concurrency rule "integrity" is accurate in the domain but harder to map to the course's characteristics than "reliability".

14. Related records

- Project README
- OpenSpec project
- Architecture report
- Kata
- Architecture characteristics
- Checkpoint A
- Checkpoint B
- Checkpoint C
- Checkpoint D
- Checkpoint E
- Findings register
- AI transparency log
- Verification records

Appendix A — Checkpoint map

Checkpoint	Course focus	Project record	Current state
A	Context and first behaviour	`checkpoint-A/README.md`	Claim, invariants and evidence established
B	Operating model and feedback	`checkpoint-B/README.md`	Hybrid lifecycle, flow evidence, decision rights
C	Quality and residual risk	`checkpoint-C/README.md`	Risk map, quality story, review decisions, residual risk
D	Architecture and changeability	`checkpoint-D/README.md`	Pre-read plus implemented idempotency change; waits L13–L16
E	Delivery, observation and recovery	`checkpoint-E/README.md`	Pre-read plus health, smoke, p95 and recovery drill; waits L17–L20

Appendix B — What this report does not claim

- It is not the final project deliverable; the semester is still running.
- It does not claim production readiness, availability or recovery.
- It does not claim that the problem is validated with real organizers.
- It does not claim independent human review.
- It does not claim that every invariant has been fault-injected.
- It does not replace the detailed checkpoint records, the findings register or the AI transparency log.

The dated report is a navigation and synthesis layer. The evidence remains in the linked records, and the human owner remains the decision owner.

15. MVP addendum — three roles, tests and evidence

The MVP implements the fan, organizer and club journeys. The methodology evidence now includes:

- backend integration tests for roles, ownership, tickets, rewards, privacy and resolution;
- 7 Playwright journeys plus serious and critical axe checks;
- secret scan and dependency audit in CI;
- a scheduled performance baseline for 20/50/100 concurrent registrations;
- MVP scope, test strategy, threat model, failure modes and ADRs 004–006.

The remaining limits are explicit: FND-007 real-user evidence and FND-008 independent review are human gates; the MVP is a course prototype, not a production-validated product. The release notes state these limits.

Appendix C — v3.0.0 addendum (2026-11-09)

This addendum records the v3.0.0 increment. It is a dated update, not a rewrite of the 2026-11-08 synthesis.

C.1 Scope and claim

The v3.0.0 increment makes the **club the paying tenant**: clubs subscribe to a simulated plan, create events, sell nominal tickets, check in attendees and read aggregate prediction insights for their own matches. Fans use one email/password account to buy and predict across clubs. A platform admin operates clubs, plans and audit. Commerce is simulated; there is no real payment provider, financial KYC, invoice, refund or settlement.

The course project remains a technical prototype. It is not validated with real clubs (FND-007); the human owner is the decision owner.

C.2 What was delivered

Increment	Evidence
Club tenancy, plans/entitlements, RLS and nominal ticketing schema	#157 / PR #158; schema verification
Shared domain contract and CLI match-day playbook	#159 / PR #160; CLI verification
Email/password auth, fan purchase, club dashboard/event page and admin console	#161 / PR #162; auth/UI verification
Member invitations/roles, fan cards, rewards, insights, export and resolution	#163 / PR #164; club UI verification
Attendee-data retention	#165 / PR #166; retention verification
Ticket QR rendering	#167 / PR #168

The work followed the approved execution plan, ADR-007–010, the capability manifest and the OpenSpec change `add-club-tenancy-and-cli`.

C.3 Verification

Check	Result
Database/application suite	42 pass / 0 fail
CLI agent scenarios	2 pass / 0 fail
Playwright/axe	11 passed
Fault injection	11 expected results
Delivery/recovery	passed
Build/security	passed

The verification records are listed in Appendix D of the architecture report and in `verification/`.

C.4 Defects found and corrected

- Legacy market resolution lost the organizer path for non-club markets; restored while adding club resolution.
- `RETURNING checked_in_at` was ambiguous with the output parameter; qualified.
- `gen_random_bytes` in a table default broke the scratch-database restore; replaced with a UUID-based token.
- Server-action success redirects were caught by `try/catch`; refactored all v3 actions to redirect after the attempt.
- Native selects failed contrast; added `bg-white` to form controls.
- CI port collisions on 54322/54324; added port cleanup to every Supabase job.

C.5 Learning

- Contract-first work paid off: the same domain wrappers serve the web UI and the CLI, and the CLI exposed authorization/state-transition gaps early.

- The database boundary is the right place for tenancy, entitlement and attendee rules; the UI is not an authorization control.
- The agent playbook is a useful course artifact because it can run the match-day journey without a browser.
- Small PRs and per-increment verification kept the v3 change reviewable despite its size.

C.6 Limits and next decisions

- No hosted deployment, external monitoring, production recovery or RTO/RPO.
- No real payments/KYC; no seat maps, wallet passes, turnstile hardware or FIGC/Questura integration.
- Attendee retention is local-only and has not had an independent legal/privacy review.
- Club privacy has not had an independent review; data resale is not implemented.
- M-INV2 remains forced-failure verified.
- FND-007 and FND-008 remain human gates.
- Checkpoints D/E finalisation still waits for L13–L16 and L17–L20; the v3 evidence is prepared in the checkpoint pre-reads.